
MEMORY CONSISTENCY MODELS

Memory Consistency Models

Topics:

- What consistency means
- Sequential consistency
- Reordering effects
- Weak models (TSO)
- Memory fences
- Programming implications

Recap: Coherence

Coherence ensures

- one value per memory location
- All cores eventually see the same value

Coherence guarantees correct value per variable, but not order of operations across variables!

Property	Guarantees
Coherence	Single variable correctness
Consistency	Global ordering

Consistency is about time/order

Example problem

Consider two variables $X = 0$ and $Y = 0$ and two cores

Core1:

$X = 1$

$r1 = Y$

Core 2:

$Y = 1$

$r2 = X$

Can we have $r1 = 0$ and $r2 = 0$?

Example problem

Sequential reasoning says it is impossible

However, it is possible in modern hardware because

- Processors reorder operations
- Writes are buffered
- Memory is not instantaneous

Sequential Consistency

Definition (Lamport): “The result is as if all operations were executed in some sequential order, and each processor’s operations appear in program order”.

The interpretation is that there are two constraints:

1. Program order preserved
2. Global interleave exists

Sequential Consistency Example

Program: C1: $X = 1 \rightarrow r1 = Y$
 C2: $Y = 1 \rightarrow r2 = X$

Under SC there are different allowed interleavings:

- C1 fully first
- C2 fully first
- Interleaved but respecting order

Sequential Consistency Example

Program: C1: $X = 1 \rightarrow r1 = Y$
 C2: $Y = 1 \rightarrow r2 = X$

Under SC there are different allowed interleavings:

- C1 fully first
- C2 fully first
- Interleaved but respecting order

Possible results:

- $r1 = 0, r2 = 1$
- $r2 = 0, r1 = 1$
- $r1 = 1, r2 = 1$

Why SC is Expensive

SC forbids:

- Instruction reordering
- Store buffering
- Many hardware optimizations

SC Kills performance!

Modern CPUs:

- Reorder instructions
- Use store buffers
- Execute out-of-order

Violates SC!

Stored Buffered Example

C1: $X = 1$ (goes to buffer)
 $r1 = Y$ (reads Y before write is visible)

C2: $Y = 1$ (goes to buffer)
 $r2 = X$ (reads X before write is visible)

Both cores read the old values

The result $r1 = 0$ and $r2 = 0$ is allowed under weak models

Hardware sees local writes first and global visibility later

Types of Reordering

Hardware may reorder:

- Load → Load
Two reads $r1 = X; r2 = Y;$
- Load → Store
Load and then write $r1 = X; Y = 1;$
- Store → Load
Store and the load $X = 1; r1 = Y;$
- Store → Store
Two stores $X = 1; Y = 1;$

Some models restrict some of these

Total Store Order (TSO)

Used in x86 architecture

Rules:

- Loads not reordered with other loads
- Stores not reordered with other stores
- Store → Load reordering allowed

Behavior:

- Writes go to store buffer
- Loads may bypass buffer

Total Store Order (TSO) - Example

Consider two cores, each one with a cache and a store buffer. At the beginning X and Y are initialized to 0

// Core 1

X = 1;

r1 = Y;

// Core 2

Y = 1;

r2 = X;

Total Store Order (TSO) - Example

Step 0:

X = 0

Y = 0

C1 buffer: empty

C2 buffer: empty

// Core 1

X = 1;

r1 = Y;

// Core 2

Y = 1;

r2 = X;

Step 1: C1 executes X = 1

C1 buffer: X = 1 (not visible to the other core)

Total Store Order (TSO) - Example

Step 2: C2 executes $Y = 1$

C1 buffer: $X = 1$

C2 buffer: $Y = 1$

```
// Core 1  
X = 1;  
r1 = Y;
```

Step 3: C1 executes $r1 = Y$

C1 reads Y from the memory

$Y = 0$ since C2 has only written in the buffer

```
// Core 2  
Y = 1;  
r2 = X;
```

Step 4: C2 executes $r2 = X$

C2 reads X from the memory

$X = 0$ since C1 has only written in the buffer

Memory Fences

To enforce order we use fence / barrier

```
X=1;  
mfence;  
r1=Y;
```

Fence ensures:

- All previous writes are visible
- No reordering across fence

Types of Fences

Full fence -> a type of CPU instruction that ensures all memory read and write operations initiated before the barrier are completed and visible to other processors before any memory operations initiated after the barrier are executed

Load fence -> a hardware instruction that ensures all load operations preceding the fence are completed and visible to the processor before any load operations following the fence

Store fence -> a hardware instruction that ensures all store operations preceding the fence are completed and visible to the processor before any store operations following the fence

Software Implications

Fences ensure correctness but reduce performance

Programmer must:

- Understand memory model
- Use synchronization primitives
- Avoid data races (two threads access same variable, at least one write and no synchronization)

Languages such as C++11 and Java define memory models which map to hardware model

Closing

Key concepts:

- Coherence $\neq$ consistency
- SC model
- Why SC is too strict
- TSO and reordering
- Memory fences

Next episode -> virtual memory